

RSA Encryption, Quantum Computers and Shor's Algorithm

Menase Teshale

RSA Algorithm for Asymmetric Encryption

Asymmetric encryption is a type of encryption where there are two keys called *public* and *private* where one (typically the private key) can only decrypt and the other (public key) can only encrypt the messages. This solves the middleman problem with symmetric encryption wherein you have to send an unencrypted key to begin communication but if the key is intercepted then the rest of the communication being encrypted is of no importance. Asymmetric encryption is vital for there to be any secure connection to the internet. It is also used in digital signatures and authentication for signing documents and commits on github, asymmetric encryption *is the reason you can be sure code for your operating system is written by the people whose job it is to write it*. The **RSA** algorithm is one of the simplest asymmetric encryption algorithms and it works as follows:

- Choose two prime numbers p, q then set $N = p \times q$
- Find $\phi(N) = (p - 1) \times (q - 1)$
- Find e such that $\gcd(e, \phi) = 1$
- Find d such that $e \times d \equiv 1 \pmod{N}$ that is the multiplicative inverse of $e \pmod{N}$
- $C \equiv M^e \pmod{N}$ therefore (N, e) is the public key
- $M \equiv C^d \pmod{N}$ therefore (N, d) is the private key

Breaking RSA on Classical Computer

This can be done by first selecting two random primes and checking if they multiply to give N if they do you can input them into the second step of the RSA algorithm to get the private and public keys. This is needlessly time consuming as you actually don't need to find the numbers themselves. You only need to find a multiple of p or q **but not both** then all you have to do is $\gcd(a, N)$ to find one of the primes (say p) then you can easily find the other $q = N/p$. If you didn't find a multiple of p or q a result in number theory guarantees that there is a number g for your number a such that $a^g = mN + 1$ for some m . We can rearrange this equation to get $(a^{\frac{g}{2}} - 1) \times (a^{\frac{g}{2}} + 1) = mN$. Now we are not guaranteed either of these factors is a number we need one of them could be a multiple of N while the other one a factor of m giving us no new information. But it is much more likely that these factors are the numbers we need 37.5% of such pairs to be exact. But the difficulty is in finding the original g to do such factorization. On classical computers we are back now to sequential guessing, this is where quantum computers come in.

Quantum Computers

Are a special type of computer that can in a single step perform a number of computations that doesn't scale linearly to the number of gates/cores/pipeline phases used to build it. They do this by using the property of superposition where a *qbit* can be in a combination of multiple states at the same time. It has to be noted that the